perplexity

I cleared both, uh, DSA. Uh, which is online assessment. Then next to online interview, one is based on DSA again, and the second was low-level design with Java. For Walmart interview, now I am appearing for my hiring manager round, which will be probably after three or four days. So I'm sharing, uh, the some information from the product team for my project. So can you give me the best HLD design, uh, which I can prepare? Um, considering it will handle the concurrency, it will introduce the caching, it will cover the core functionality, a few of the core functionality, which impact on the hiring manager. Which I explain, and also give me the reason of which component why you choose that, like, uh, SQL or Cassandra or PostgreSQL, why you choose that, and which caching you use, and which methodology in caching on eviction policy use. So I basically, I want a detailed diagram how you will diagram, which I can learn and prepare for HLD round, basically.

You can present this as a **Workforce Management (WFM) Attendance & Leave Platform** that supports high concurrency, strong correctness for balances, and low latency for managers and employees. Below is a complete, interview-ready HLD you can study and adapt in your hiring-manager round.[1]

1. Problem clarification

Functional requirements

- Employee attendance: time-punches from mobile/web/time-clock devices, punch edits, view timesheets.
- Leave management: apply for leave, approve/reject, accruals and balance tracking.
- Concurrency case: leave request and punches arriving at the same time must not corrupt balances and attendance.
- Notifications: alert employees/managers on approvals, exceptions, anomalies.
- Reporting : query attendance, leave history, audit logs.
- Multi-tenant: multiple enterprises on same platform, strict data isolation.

Non-functional requirements

- High availability (target 99.9%+).
- Low latency (< 150 ms P99 for core APIs under normal load).
- Horizontally scalable to millions of employees.
- Strong consistency for balance changes, eventual consistency for analytics/search.
- Auditability and compliance (immutable logs for punches, approvals, changes).

Scale assumptions (for back-of-envelope)

- 10M registered employees, 1M DAU.
- Peak 200K punch events/minute (≈3.3K/s).
- Peak 100K leave requests/day (≈1.2/s, but bursty during mornings).
- 5 years of history retained, ~50–100 GB of hot OLTP data per tenant, TBs of cold data in warehouse.

2. Capacity estimation (quick)

Assume:

- Each punch or leave request = 1 KB payload after normalization.
- Punches: 3.3K/s → ~3.3 MB/s → ~285 GB/day raw, but most gets aggregated/archived.
- Audit & events: each action generates 1–3 extra events → 3x traffic on event bus.
- Read:write ratio around 5:1 (lots of dashboard and mobile views versus updates).

Use these only as ballpark numbers to justify choices (Kafka, sharding, cache).

3. High-level architecture

Describe it top-down:

1. **Clients**
   - Web portal (React/Angular) for managers and HR.
   - Mobile app (Android/iOS) for employees.
   - Physical time-clock / IoT devices.
2. **Edge layer**
   - CDN for static assets.
   - API Gateway + WAF for auth, rate limiting, routing, tenant isolation.
3. **Service layer (stateless microservices)**
   - Employee Context Service.
   - Attendance API (punches/edits).
   - Leave API & Approval Workflow Service.
   - Schedule Service (shifts/patterns).
   - Accrual Service (balances and grants).
   - Rules Engine (attendance & leave policies).
   - Notification Service (email, SMS, push).
   - Reporting/Query API.
   - Admin/Config Service.
   - Audit Service, Fraud/Anomaly Detection.
4. **Async event layer**
   - Kafka (or Pulsar/RabbitMQ) for punches, leave events, approvals, balance updates, notification events, audit events.
   - Worker pool / background consumers for recalculation, exports, enrichment, aggregation.
5. **Data layer**
   - Transactional DBs (PostgreSQL per service or per bounded context).
   - Redis cache cluster for hot data (employee context, balance snapshots).
   - Search index (Elasticsearch/OpenSearch) for queries and audit lookup.
   - Object storage (S3/GCS/Azure Blob) for exports, reports, archives.
   - Data warehouse (BigQuery/Snowflake/Redshift) for analytics.
   - Centralized Observability stack (logs, metrics, traces).
   - Security/governance services (RBAC, tenant isolation, encryption, DLP).

4. Architecture diagram (Mermaid)

You can paste this directly into Mermaid Live or any markdown tool that supports Mermaid and walk the interviewer through it.

```
flowchart TD

subgraph Clients
  W[Web / Manager UI]
  M[Mobile App]
  T[Timeclock Devices]
end

W --> AG
M --> AG
T --> AG

subgraph Edge
  CDN[CDN]
  AG[API Gateway / WAF<br/>Auth, Rate Limit, Tenant Routing]
end

CDN <-- Static Assets --> W

AG -->|sync| SvcLayer
AG -->|async callbacks / webhooks| Notif

subgraph SvcLayer[Stateless Service Layer]
  EC[Employee Context Service]
  ATT[Attendance API<br/>Punches / Edits]
  LV[Leave API<br/>Requests]
  SCH[Schedule Service<br/>Shifts / Patterns]
  ACC[Accrual Service<br/>Balances / Grants]
  WF[Approval Workflow]
  RE[Rules Engine<br/>Attendance Eval]
  NQ[Reporting / Query API]
  ADM[Admin / Config Service]
  AUD[Audit Service]
  FRAUD[Fraud / Anomaly Detection]
  NOTIF[Notification Service]
end

ATT -->|events| EB
LV -->|events| EB
WF -->|events| EB
ACC -->|events| EB

ATT --> EC
LV --> EC
LV --> ACC
ATT --> RE
RE --> ACC
SCH --> ATT
SCH --> LV
```

```
subgraph EB[Async Event Layer]
  K[Kafka / Event Bus<br/>Topics: punches, requests, approvals, balances, notifications,
end

K --> WP
K --> SP

subgraph Workers[Worker Pool / Consumers]
  WP[Recalc / Notify / Export / ETL]
  SP[Stream Processing<br/>Enrichment / Aggregates]
end

Workers --> DataLayer
SvcLayer --> DataLayer

subgraph DataLayer[Data Layer]
  subgraph OLTP[Transactional DBs]
    PG_ATT[(Postgres: Attendance)]
    PG_LEAVE[(Postgres: Leave)]
    PG_ACC[(Postgres: Accruals)]
  end

  OUTBOX[(Outbox Tables)]
  REDIS[(Redis Cluster<br/>Hot Balances, Context)]
  SEARCH[(Elasticsearch<br/>Requests / Audit lookup)]
  OBJ[(Object Storage<br/>Reports / Archives)]
  WH[(Data Warehouse<br/>BI / Trends)]
end

ATT --> PG_ATT
LV --> PG_LEAVE
ACC --> PG_ACC

PG_ATT --> OUTBOX
PG_LEAVE --> OUTBOX
PG_ACC --> OUTBOX
OUTBOX --> K

EC --> REDIS
ACC --> REDIS
NQ --> SEARCH
WP --> OBJ
SP --> WH

subgraph Ops[Platform]
  OBS[Observability<br/>Logs / Metrics / Traces]
  SEC[Security / Governance<br/>RBAC, Encryption, DLP]
end

SvcLayer --> OBS
EB --> OBS
DataLayer --> OBS
AG --> SEC
SvcLayer --> SEC
```

5. Deep dive into core components

API Gateway / WAF

- Terminates TLS, authenticates via JWT/OAuth2, enforces RBAC and tenant routing.
- Handles rate limiting, request validation, and protects from DDoS.
- Good choices: Kong, NGINX, AWS API Gateway, Apigee.

Stateless Service Layer

Reasoning: stateless makes it easy to autoscale and handle spikes; state is kept in databases, Redis, and Kafka.

Key services and how they interact:

1. **Employee Context Service**
   - Maintains employee profile, employment status, manager hierarchy, tenant mapping.
   - Cached aggressively in Redis (keyed by employeeId+tenant).
   - Used by Attendance, Leave, Accrual, Rules Engine.
2. **Attendance API**
   - Accepts punches from all devices.
   - Performs basic validation (shift boundaries from Schedule Service, duplicate detection using idempotency key).
   - Writes to Attendance DB using ACID transactions (PostgreSQL), then puts an event into an Outbox table.
   - A background process publishes from Outbox to Kafka (exactly-once-ish pattern).
3. **Leave API & Approval Workflow**
   - Leave service validates employee and window, then calls Accrual Service to reserve leave.
   - Workflow engine manages multi-step approvals, reminders, and SLA violations using Kafka topics and timers.
   - When approved, reservation becomes final deduction; when rejected/cancelled, reservation is released.
4. **Accrual Service**
   - Maintains leave balances, accrual rules, and grants.
   - Uses strong consistency via PostgreSQL transactions; each balance update row has version/lock (optimistic locking).
   - Exposes two operations: `ReserveBalance` (temporary hold) and `CommitReservation`.
   - Writes compact balance snapshots to Redis for fast reads and to avoid double-spend.
5. **Rules Engine**

- Evaluates attendance policies (late, overtime, breaks, etc.).
- Consumes punch events from Kafka and looks up context (schedule, leave, historical punches) from Redis/DB.
- Emits evaluated events (e.g., "Late by 10 minutes", "Overtime 2 hours") to downstream services.

6. **Notification Service**
   - Consumes events such as `LeaveApproved`, `ExceptionDetected`.
   - Sends email/SMS/push using provider integrations, stores delivery status.
   - Stateless with retry/backoff via Kafka consumer groups.
7. **Audit Service & Fraud/Anomaly Detection**
   - Every important state change writes to immutable audit store (append-only table or log).
   - Anomaly detection consumes streams to flag suspicious punch patterns, bulk edits, or policy violations.
8. **Reporting / Query API**
   - Serves managers and HR dashboards.
   - For heavy analytics it reads from warehouse; for recent data, from PostgreSQL plus ElastiSearch for text and filters.

Async Event Layer (Kafka)

Why Kafka:

- High throughput, partitioned for horizontal scaling, durable log for replay, good ecosystem.
- Decouples write path (fast, low latency) from heavy processing (recalc, notifications, analytics).

Key topics:

- `punch-events`, `leave-requests`, `leave-approved`, `balance-updates`, `notification-commands`, `audit-events`.

Worker Pool / Stream Processing

- Workers perform slow/expensive tasks: attendance recalculation for retroactive changes, accrual recomputation, exports, anomaly detection.
- Stream processors (Kafka Streams / Flink / Spark Streaming) maintain derived aggregates such as daily hours per employee or per team to support dashboards.

6. Database design and choices

Why PostgreSQL for OLTP

- Strong relational guarantees (ACID) required for financial-like data (leave balances, payroll-related attendance).
- Mature indexing, partitioning, and transactional features.
- Fits typical WFM relational schemas: employees, punches, shifts, balances.

You can mention that Cassandra or DynamoDB are great for write-heavy, ultra-large scale workloads but complicate strong consistency and transactions; for a WFM system where correctness of balances is critical, **PostgreSQL is a safer first choice**. You can always introduce Cassandra later for event or metric storage.

Example schemas (simplified)

Attendance DB:

- `employees(id, tenant_id, manager_id, status, ...)`
- `punches(id, employee_id, tenant_id, punch_time_utc, type, source, shift_id, created_at, idempotency_key, ...)`
- Indexes: `(employee_id, punch_time_utc)`, `(tenant_id, punch_time_utc)`, unique on `idempotency_key` per tenant.

Leave DB:

- `leave_requests(id, employee_id, tenant_id, start_date, end_date, type, status, created_at, updated_at, ...)`
- `leave_approvals(id, request_id, approver_id, status, decided_at, ...)`
- Indexes on `(employee_id, start_date)`, `(tenant_id, status)`.

Accrual DB:

- `leave_balances(id, employee_id, tenant_id, type, balance, reserved, version, updated_at, ...)`
- `accrual_rules(id, tenant_id, type, frequency, amount, ...)`
- Use optimistic locking: update `leave_balances` where `version = ?`, increment version.

Audit DB:

- `audit_log(id, entity_type, entity_id, tenant_id, actor_id, action, payload_json, created_at)`
- Append-only, partitioned by date/tenant.

Sharding strategy

- **Primary shard key: tenant_id** (enterprise).
- Within a tenant, large tables (punches, audit_log) partitioned by time (month or quarter).
- Each shard is a separate PostgreSQL cluster; routing handled by a small "Tenant Router" or via connection strings stored in a config service.

This lets you scale by onboarding tenants into different clusters and keeps hot tenants from impacting others.

7. Caching strategy

Why Redis

- In-memory, low-latency, supports data structures (hashes, sets) and TTLs.
- Easily clustered and replicated, widely used for session and data caching.

What to cache

- Employee context: profile, manager, employment status.
- Leave balance snapshot for each employee & leave type.
- Policy metadata: rules, calendars, holidays.
- Schedule lookups for upcoming weeks.

Example key design:

- `emp:{tenantId}:{empId}` → JSON blob, TTL 5–15 minutes.
- `leaveBal:{tenantId}:{empId}:{type}` → `{balance, reserved, version}`, TTL 1–5 minutes, invalidated on update.

Caching patterns and eviction policy

- **Write-through + explicit invalidation** for balances:
  - On update, write to DB in a transaction.
  - On success, update Redis with new snapshot (or delete key so next read repopulates).
  - This avoids stale cached balances for critical operations.
- **Read-through (lazy) cache** for employee context/schedules:
  - If key missing, service reads from DB, writes back to Redis, and returns.
- **Eviction policy:**
  - Use `allkeys-lru` or `volatile-lru` depending on how much you rely on TTLs.
  - For hot keys like balances, also use reasonable TTL (e.g., 5–10 minutes) and manual invalidation on writes.

You can explain that LRU is a good default because access patterns are skewed towards recently active employees/managers.

8. Handling concurrency (key use case)

Scenario: employee file leave request while punches arrive from timeclock at the same time.

Present this step-by-step to the hiring manager.

1. **Leave reservation path**
   - Employee calls `POST /leave-requests`.
   - Leave API validates inputs and calls Accrual Service `ReserveBalance(empId, type, amount)` inside a DB transaction.
   - Accrual Service: `SELECT ... FOR UPDATE` row from `leave_balances`.
   - If `balance - reserved >= amount`, it increases `reserved` and writes an event `LeaveReserved` into outbox; else rejects.
   - Commit transaction → outbox publisher sends `LeaveRequested` + `BalanceReserved` events to Kafka.
2. **Punch processing path**
   - Timeclock submits punch to Attendance API.
   - Attendance API uses idempotency key to dedupe duplicates, writes punch row in `punches` table in PostgreSQL, commits, then publishes `PunchCreated` via outbox.
3. **Rules Engine & Accrual**
   - Rules Engine consumes `PunchCreated`, computes worked hours and overlaps with any reserved leaves.
   - It may trigger additional adjustments (e.g., if employee actually worked during planned leave).
4. **Approval flow**
   - Manager approves leave via Workflow Service.
   - Workflow publishes `LeaveApproved` event, which triggers Accrual Service `CommitReservation` for that request:
     - Another transaction on `leave_balances` row deducts from `balance`, decreases `reserved`, and writes `BalanceUpdated` event.
5. **Correctness guarantees**
   - Use **row locks or optimistic locking** on `leave_balances` so only one concurrent update succeeds; others retry.
   - Idempotency keys on punches and leave requests prevent duplicates.
   - Exactly-once-ish messaging via outbox pattern ensures events are never lost or duplicated relative to DB state.

- Reads that must be strongly consistent (e.g., when creating a new reservation) can read from DB; normal dashboard views can read cached snapshots from Redis.

You can summarize: correctness is guaranteed by ACID transactions + locking in PostgreSQL, plus idempotency and outbox for events; scalability comes from stateless services + Kafka + Redis.

9. API design (core endpoints)

You do not need every endpoint; just a representative set showing how you think.

```
POST   /api/v1/punches
GET    /api/v1/punches?employeeId=&from=&to=
POST   /api/v1/leave-requests
GET    /api/v1/leave-requests/{id}
POST   /api/v1/leave-requests/{id}/approve
POST   /api/v1/leave-requests/{id}/reject
GET    /api/v1/balances?employeeId=&type=
GET    /api/v1/schedules?employeeId=&from=&to=
GET    /api/v1/audit?entityType=&entityId=
```

Mention that APIs are authenticated via JWT containing `tenant_id`, `user_id`, and roles; gateway injects this into headers for internal services.

10. Scaling strategy

- Scale all stateless services horizontally behind a load balancer.
- Partition Kafka topics by tenant or employee hash to scale consumption.
- Use connection pooling and read replicas for PostgreSQL (reads from replicas, writes to primary).
- For very large tenants, allocate dedicated DB clusters and Kafka partitions.
- Autoscale workers based on queue lag and CPU.

11. Fault tolerance and consistency model

- Each DB cluster uses synchronous replication within AZ, async to cross-region replica.
- Kafka cluster with replication factor 3, min ISR 2.
- Services use circuit breakers and retries with backoff when calling each other.
- Consistency:
  - Strong for core OLTP operations (balances, punches).
  - Eventual for projections (reports, search, analytics).
- In case of partial failure (e.g., notification provider down), events remain in Kafka and are retried.

12. Security & observability

- Security:
  - OAuth2/OIDC for identity, RBAC per tenant (employee, manager, HR, admin).
  - Data encrypted at rest (KMS-managed keys) and TLS in transit.
  - Field-level masking for sensitive data; DLP on exports.
- Observability:
  - Structured logs (JSON) shipped to ELK/Cloud Logging.
  - Metrics (latency, error rate, queue lag, DB connections) with Prometheus + Grafana dashboards.
  - Distributed tracing (Jaeger/Zipkin/OpenTelemetry) to see cross-service flows.

13. Key trade-offs you can articulate

Use these sentences in the interview to sound senior:

Decision	Option chosen	Why it fits this system
DB for balances	PostgreSQL (relational)	Strong consistency and transactions; simpler semantics than Cassandra for financial-like balances
Event backbone	Kafka	High throughput, partitioning, replay, decouples services, good for spikes in punches
Cache	Redis	Low latency, rich data structures, widely used; enables offloading DB for reads
Service style	Stateless microservices	Independent scaling, clear bounded contexts (attendance, leave, accruals)
Sharding	Tenant-based	Natural multi-tenant boundary; keeps large customers isolated
Consistency	Strong in OLTP, eventual elsewhere	Protects balances while allowing scalable analytics and reporting

When asked “what if scale doubles or triples”, talk about:

- Adding more Kafka partitions and consumer groups.
- Sharding tenants to additional Postgres clusters.
- Moving cold data to warehouse, keeping OLTP lean.
- Introducing CQRS: separate write models and read projections for heavy dashboards.

14. Best interview summary notes (for quick revision)

Use this as a final one-page recap before your round:

Key components

- API Gateway + WAF.
- Stateless services: Attendance, Leave, Accruals, Employee Context, Rules Engine, Workflow, Notification, Audit, Reporting.
- Kafka event bus + worker pool.
- PostgreSQL per bounded context, tenant-sharded.
- Redis for hot data, Elasticsearch for search, data warehouse + object storage for analytics and exports.

Key scaling techniques

- Stateless microservices behind load balancers.
- Kafka for burst absorption and asynchronous processing.
- Tenant-based sharding and time-partitioned tables in Postgres.
- Redis caching and read replicas for high read throughput.

Key trade-offs

- Chose relational DB over Cassandra to keep strong correctness for leave balances.
- Chose eventual consistency for reports to keep core path fast and simple.
- Used write-through + invalidation cache for balances to avoid stale reads.
- Used outbox pattern to ensure DB and events are consistent.

If you tell me roughly how many services you're comfortable explaining in detail during a 45–60 minute discussion, I can trim this into a shorter "version 1" you can memorize more easily.

⁂

1. IMG_5801.jpeg
2. IMG_5802.jpeg
3. IMG_5800.jpeg
4. IMG_5804.jpeg
5. IMG_5803.jpeg
6. IMG_5799.jpeg
7. pasted-text.txt
8. pasted-text.txt